



# ارتقای معماری پردازنده از تک چرخه به خط لوله پنج مرحله ای

معماری کامپیوتر – گزارش کار تمرین عملی هفتم

استاد

دکتر سربازی آزاد

دانشگاه صنعتی شریف

بهار ۱۴۰۵

محمد مهدی مرادی – ۴۰۳۱۰۶۶۸۱

## فهرست

|   |                                                            |
|---|------------------------------------------------------------|
| ۳ | مقدمه.....                                                 |
| ۳ | ۱-مروری بر معماری پایه و حافظه نهان.....                   |
| ۳ | ۲- ارتقای پردازنده به معماری خط لوله ۵ مرحله‌ای.....       |
| ۵ | ۳- مدیریت مخاطرات (Hazards) در خط لوله.....                |
| ۶ | ۴- چالش‌های محیط شبیه‌ساز، باگیابی و راهکارهای مهندسی..... |
| ۶ | نتیجه گیری.....                                            |

## مقدمه:

هدف از این مجموعه پروژه‌ها، طراحی یک پردازنده کامل ۳۲ بیتی مبتنی بر معماری MIPS از پایه و با استفاده از زبان توصیف سخت‌افزار Verilog بود. مسیر طراحی از یک پردازنده تک‌چرخه (Single-Cycle) در تمرین پنجم آغاز شد. سپس در تمرین ششم، برای کاهش زمان دسترسی به حافظه، حافظه نهان (L1 Cache) طراحی و به معماری افزوده شد. در نهایت، در تمرین هفتم، برای افزایش توان عملیاتی (Throughput) و بهره‌وری، معماری پردازنده به یک خط لوله ۵ مرحله‌ای (۵-Stage Pipeline) ارتقا یافت. در تمامی مراحل، محدودیت‌های طراحی (مانند عدم استفاده از عملگرهای آماده ضرب و تقسیم) و چالش‌های هماهنگی با تست‌بنچ ارزیاب با موفقیت پشت سر گذاشته شد.

## ۱-مروری بر معماری پایه و حافظه نهان:

پایه‌های اصلی پردازنده خط لوله، بر روی ماژول‌های مستحکمی بنا شد که در تمرین‌های پیشین طراحی شده بودند:

**الف) محاسبات ریاضی بدون عملگر آماده (ALU, MUL, DIV):**

برای رعایت محدودیت پروژه، واحدهای ضرب و تقسیم به صورت ماشین‌های حالت (State Machine) کلاک‌دار طراحی شدند که عملیات را در ۳۲ چرخه کلاک (با استفاده از الگوریتم‌های Shift-and-Add و Restoring Division) انجام می‌دهند.

**ب) طراحی L1 Cache با FSM هفت‌وضعیتی:**

برای برقراری ارتباط با حافظه اصلی و مدیریت تاخیرها، یک ماشین حالت ۷ وضعیتی (IDLE, READ\_REQ, READ\_WAIT, ALLOC\_REQ, ALLOC\_WAIT, WRITE\_REQ, WRITE\_WAIT) طراحی شد. با تاکید تی ای، درخواست‌های خواندن و نوشتن به حافظه اصلی تنها به شکل یک پالس ۱ کلاکه ارسال شدند.

**ج) سیاست Write-Allocate:**

در زمان رخ دادن Write Miss در حافظه نهان، ابتدا کل بلاک ۱۲۸ بیتی از حافظه واکشی شده، داده جدید در آن جایگذاری شده و سپس بلاک آپدیت‌شده در حافظه اصلی نوشته می‌شود.

## ۲- ارتقای پردازنده به معماری خط لوله ۵ مرحله‌ای:

برای افزایش چشمگیر توان عملیاتی و کاهش زمان تناوب کلاک (Clock Cycle Time)، مسیر داده پردازنده شکافته شد و به ۵ مرحله‌ی کاملاً ایزوله ارتقا یافت. برای حفظ انسجام داده‌ها، ۴ رجیستر واسط (Pipeline Registers) شامل F\_D, D\_E, E\_M و M\_W پیاده‌سازی شدند که علاوه بر داده‌ها، سیگنال‌های کنترلی را نیز کلاک به جلو می‌برند. در هر بافر، یک بیت اعتبار (Valid Bit) تعبیه شد تا دستورات واقعی از حباب‌ها (NOP) متمایز شوند. جزئیات پیاده‌سازی هر مرحله به شرح زیر است:

## الف) مرحله (F) Instruction Fetch

در این مرحله، آدرس موجود در ثبات PC (پس از تبدیل از آدرس بایتی به کلمه‌ای با شیفت به راست) به حافظه دستورات ارسال می‌شود. همچنین در همین مرحله، مقدار آدرس بعدی (Next\_PC) از طریق یک مالتی‌پلکسر چندورودی انتخاب می‌شود که گزینه‌های آن شامل  $PC+4$  و آدرس‌های پرش (محاسبه‌شده در مرحله Decode) است. اگر خط لوله دچار توقف (Stall\_F\_D) نشود، دستور خوانده‌شده به همراه آدرس PC و بیت  $Valid=1$  در بافر F\_D ثبت می‌شود. در صورت وقوع پرش، سیگنال Flush\_F صادر شده و دستور واکنشی‌شده به یک حباب ( $Valid=0$ ) تبدیل می‌گردد.

## ب) مرحله (D) Instruction Decode و حل زود هنگام پرش‌ها:

علاوه بر رمزگشایی دستور در واحد کنترل (CU) و خواندن مقادیر از بانک ثبات‌ها (RF)، یکی از خلاقانه‌ترین بخش‌های معماری ما «بررسی زود هنگام پرش‌ها» (Early Branch Resolution) در این مرحله است. برای کاهش هزینه (Penalty) پرش‌ها از ۳ کلاک به ۱ کلاک، محاسبه‌ی آدرس هدف انشعاب (Branch Target) و آدرس Jump، و همچنین بررسی شرط برابری ثبات‌ها ( $fw\_rs\_D == fw\_rt\_D$ ) مستقیماً در همین مرحله انجام می‌شود. سیگنال‌های کنترلی و مقادیر خوانده‌شده (پس از عبور از شبکه Forwarding مرحله D) در بافر D\_E ذخیره می‌شوند.

## ج) مرحله (E) Execute

در این مرحله، عملیات اصلی ریاضی و منطقی توسط ALU و مازول‌های مستقل MUL/DIV انجام می‌پذیرد. عملوندهای ALU از طریق یک شبکه مسیره‌ی پیشرفته (فورواردینگ) تامین می‌شوند تا بروزترین مقادیر به ALU برسد. اگر دستور به محاسبات طولانی مدت ضرب یا تقسیم نیاز داشته باشد، یک پالس شروع (alu\_start\_pulse) تولید شده و تا زمان یک شدن سیگنال alu\_ready، مراحل قبلی خط لوله فریز می‌شوند.

## د) مرحله (M) Memory

دستوراتی که به حافظه داده نیاز دارند (LW/SW) در این مرحله عمل می‌کنند. برای جلوگیری از خطاهای محیط شبیه‌ساز (تست‌بنچ)، آدرس ارسالی به حافظه در این مرحله فیلتر می‌شود؛ بدین معنا که اگر سیگنال‌های MemRead و MemWrite صفر باشند، یک آدرس کاملاً امن ( $h'000040032$ ) به بیرون ارسال می‌شود تا از هشدارهای Invalid Address جلوگیری گردد.

## ه) مرحله (W) Write Back

در نهایت، با استفاده از یک مالتی‌پلکسر ۴ به ۱ (با کنترل سیگنال MemtoReg)، تصمیم‌گیری می‌شود که چه داده‌ای باید در بانک ثبات‌ها نوشته شود. این داده می‌تواند نتیجه ALU، داده خوانده‌شده از حافظه، آدرس  $PC+4$  (برای دستور JAL) و یا مقدار تولید شده برای دستور LUI باشد.

### ۳- مدیریت مخاطرات (Hazards) در خط لوله:

با توجه به ماهیت اجرای همزمان دستورات، وابستگی‌های متعددی در پردازنده رخ می‌دهد که با طراحی شبکه‌های گذرگاه فرعی (Forwarding) و توقف‌های هوشمند (Smart Stalling) بهینه‌سازی شدند:

الف) مخاطرات داده (Data Hazards) و شبکه Forwarding دوگانه:

برای حل وابستگی‌های داده‌ای (Read-After-Write)، به جای توقف خط لوله، مقادیر جدید قبل از نوشته شدن در ثبات‌ها روبروده می‌شوند. ما دو شبکه فورواردینگ کاملاً مجزا پیاده‌سازی کردیم:

۱. فورواردینگ برای مرحله `Execute (fw_rs_E, fw_rt_E)`: اگر دستوری در مرحله E به داده‌ای نیاز داشته باشد که در مرحله M (کلاک قبلی) یا مرحله W (دو کلاک قبلی) تولید شده است، مالتی پلکسرها مقادیر `E_M_ALUOut` یا `M_W_Data_to_Reg` را مستقیماً به ورودی ALU هدایت می‌کنند. اولویت همواره با مرحله M (جدیدترین داده) است.

۲. فورواردینگ برای مرحله `Decode (fw_rs_D, fw_rt_D)`: از آنجایی که شرط پرش‌ها (Branch) در مرحله D بررسی می‌شود، ما به داده‌های به‌روز در همین مرحله نیاز داریم. لذا یک شبکه فورواردینگ ثانویه طراحی شد تا مقادیر تولید شده در مراحل M و W را به مقایسه‌گر انشعاب در مرحله D برساند.

ب) مخاطرات کنترل (Control Hazards) و منطق Flushing:

همان‌طور که اشاره شد، پرش‌ها در مرحله D حل می‌شوند. اگر پردازنده تشخیص دهد که یک پرش (Branch Taken) یا پرش مطلق (Jump/JR) رخ داده است، سیگنال `Flush_F` فعال می‌شود. این سیگنال باعث می‌شود دستوری که در همان کلاک وارد مرحله Fetch شده (و اشتباه بوده) فوراً پاک شود ( $F\_D\_Valid \leq 0$ ) و از ورود آن به مراحل بعدی جلوگیری به عمل آید.

ج) مدیریت توقف‌های هوشمند (Smart Stall Logic):

توقف (Freeze) بی‌مورد باعث افت راندمان می‌شود؛ لذا منطق Stall ما کاملاً شرطی و مبتنی بر وابستگی‌های واقعی (True Dependencies) طراحی شد. سیگنال `Stall_F_D` (که PC و بافر `F_D` را متوقف می‌کند و حباب به بافر `D_E` می‌فرستد) در سه حالت فعال می‌شود:

- توقف ساختاری (`Stall_ALU`): در هنگام اجرای دستورات `MUL` و `DIV` که ۳۲ کلاک زمان می‌برند.
- خطای `Load-Use (Stall_LoadUse)`: اگر دستور موجود در مرحله E از نوع خواندن از حافظه (`MemRead`) باشد و دستور موجود در مرحله D به ثبات مقصد آن نیاز داشته باشد، پردازنده ۱ کلاک متوقف می‌شود. برای بهینه‌سازی، سیگنال‌های `D_Rs_used` و `D_Rt_used` به کنترلر استال اضافه شدند تا فقط زمانی استال رخ دهد که دستور مرحله D واقعاً از ثبات‌های `rs` یا `rt` استفاده می‌کند (جلوگیری از استال‌های کاذب).
- استال مربوط به پرش (`Stall_Branch`): از آنجا که پرش در مرحله D بررسی می‌شود، اگر دستور بلافاصله قبل از پرش (که اکنون در مرحله E است) در حال محاسبه‌ی مقداری باشد که شرط پرش به آن وابسته است، چون نتیجه هنوز آماده نیست و نمی‌تواند به مرحله D فوروارد شود، پردازنده به صورت خودکار یک کلاک متوقف می‌شود تا محاسبه‌ی ALU تمام شده و در کلاک بعدی (از مرحله M) به مرحله D فوروارد گردد.

## ۴- چالش‌های محیط شبیه‌ساز، باگ‌یابی و راهکارهای مهندسی

بخشی از زمان این پروژه، صرف همگام‌سازی معماری پردازنده با الزامات تست‌بنچ شد. راهکارهای زیر برای رفع این چالش‌ها اتخاذ گردید:

### الف) طراحی گذرگاه داخلی ثبات‌ها (Write-Through RF Bypass):

در تست‌بنچ، دقیقاً در لحظه‌ای که سیگنال W\_Valid یک می‌شد (و هنوز لبه بالارونده کلاک بعدی فرا نرسیده بود)، وضعیت ثبات‌ها برای اعتبارسنجی خوانده می‌شد که منجر به خطای "FAILED at step ۰" می‌گردید. برای رفع این مشکل، بانک ثبات‌ها (RF) به صورت Write-Through طراحی شد؛ یعنی مقداری که قرار است در ثبات نوشته شود، به صورت ترکیبیاتی (Combinational) مستقیماً روی پورت‌های خروجی  $R_0$  تا  $R_{31}$  قرار می‌گرفت تا شبیه‌ساز در همان سیکل بتواند مقدار صحیح را ببیند.

## نتیجه‌گیری

تبدیل یک پردازنده تک‌چرخه به معماری خط لوله، نیازمند دقت بالا در زمان‌بندی و انتقال وضعیت دستورات است. طراحی ماژولار پایپ‌لاین، بهره‌گیری از تکنیک‌های Forwarding کامل در مراحل مختلف، و هندل کردن هوشمندانه پرش‌ها در مرحله Decode باعث شد پردازنده با بالاترین کارایی ممکن عمل کند. غلبه بر چالش‌های تست‌بنچ (از جمله پیاده‌سازی مکانیزم Write-Through و ایمن‌سازی آدرس‌های حافظه) نشان داد که پردازنده طراحی شده نه تنها از نظر تئوری کامل است، بلکه انعطاف لازم برای تعامل با محیط‌های شبیه‌ساز سخت‌گیرانه را نیز داراست. اجرای موفق برنامه تبدیل مبنای ۳ و سری فیبوناچی با وضعیت ACCEPTED، گواه صحت و پایداری این معماری است. در شکل زیر نیز خروجی گرفته شده از تست بنچ را مشاهده می‌کنید که هر ۲۰۸ تست کیس را accept شده است.

```
mahdi@mahdi-ASUS-Zenbook-14:~/Documents/CA/HWs/Practicals/ca4042-judge$ ./validate-verilog.sh ../7/HW7-403106681.v HW7/tb.v
WARNING: Invalid address to read : xxxxxxxX

fib(10) = exp: 55  real: 55
fib(11) = exp: 89  real: 89
product = exp: 4895 real: 4895
base3 digits LSD first:
2 2 0 1 0 2 0 2
base3 = 20201022
ACCEPTED
208 / 208
HW7/tb.v:469: $finish called at 1630 (1s)
```